Apache NiFi Assignment
Real-Time Data Ingestion & Processing

Pipeline Documentation

Objective: Design and implement a complete data ingestion pipeline using Apache NiFi to simulate real-time data, process it, and store it in a distributed system (HDFS).

1 High-Level Architecture

The pipeline utilizes a decoupled architecture, extracting data from two distinct sources: a real-time messy JSON file stream and a clean operational PostgreSQL database. The data is ingested, standardized, enriched, and ultimately loaded into a Hadoop Distributed File System (HDFS).

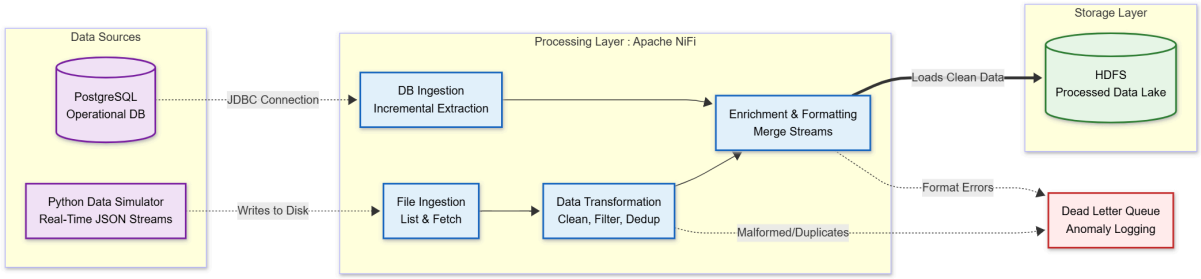

Figure 1: High-Level Data Pipeline Architecture showing Source, Processing, and Storage layers.

1.1 Core Components

- **Data Sources:** - A Python simulator generating semi-structured, messy e-commerce JSON data (with nulls, duplicates, and inconsistent dates) continuously to a local disk.
  - A PostgreSQL operational database tracking order statuses.
- **Processing Layer (Apache NiFi):** Handles stateful ingestion, in-memory deduplication, SQL-based filtering, format conversion, and stream merging.
- **Storage Layer (HDFS):** The final Data Lake destination for the processed records.
- **Error Handling (DLQ):** A Dead Letter Queue pattern implemented across all processing stages to trap and log anomalies without dropping data.

2 NiFi Implementation & Process Groups

To maintain a clean and enterprise-ready canvas, the pipeline is organized into distinct Process Groups based on their functional responsibilities.

2.1 Overall Pipeline View

The root canvas handles the routing between the ingestion, transformation, and enrichment phases.

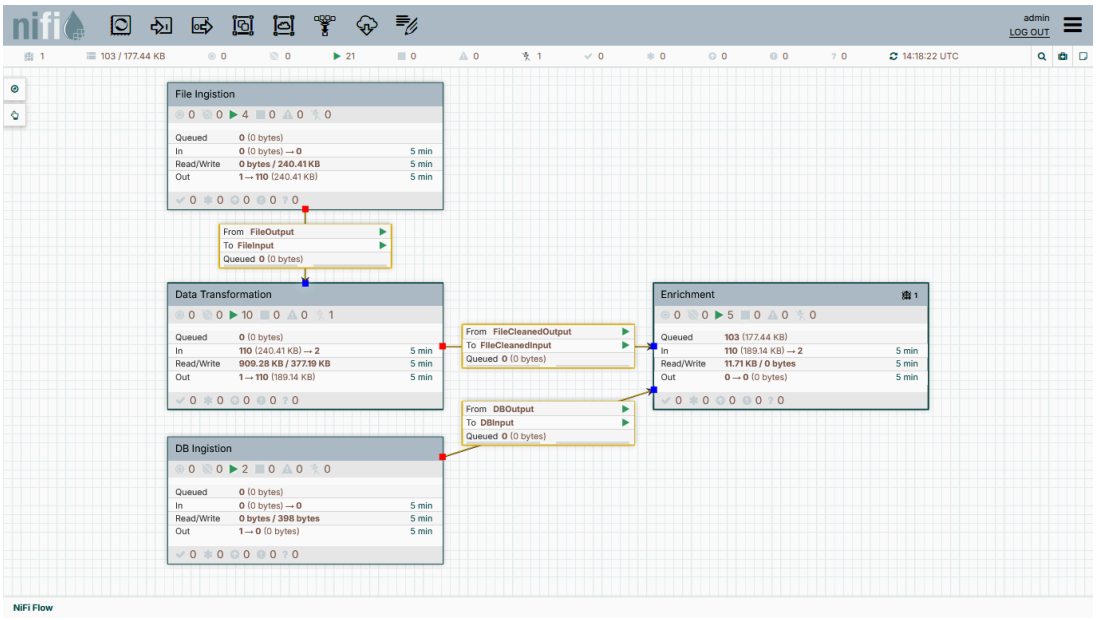

Figure 2: The root NiFi canvas displaying decoupled Process Groups.

2.2 Phase 1: File Ingestion

This group handles the messy real-time JSON data using the mandated `ListFile` → `FetchFile` pattern.

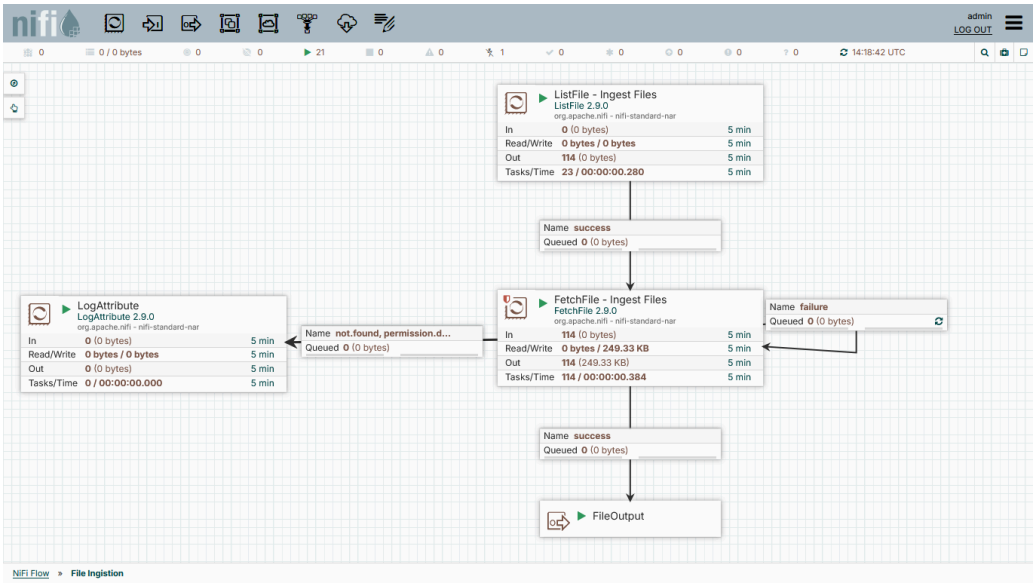

Figure 3: File Ingestion leveraging stateful listing and penalty loops.

- **Design Decision:** `ListFile` is configured to run every 5 seconds to manage disk I/O efficiently while natively maintaining state to prevent duplicate file ingestion.
- **Reliability:** If `FetchFile` encounters a disk read error (`comms.failure`), the flowfile is routed back into the processor with **Penalization** enabled to prevent infinite, aggressive retry loops. Missing files are routed to an anomaly log (DLQ).

2.3 Phase 2: Data Transformation

This group applies rigorous data quality rules to the semi-structured JSON stream using NiFi's high-performance Record-Oriented processors.

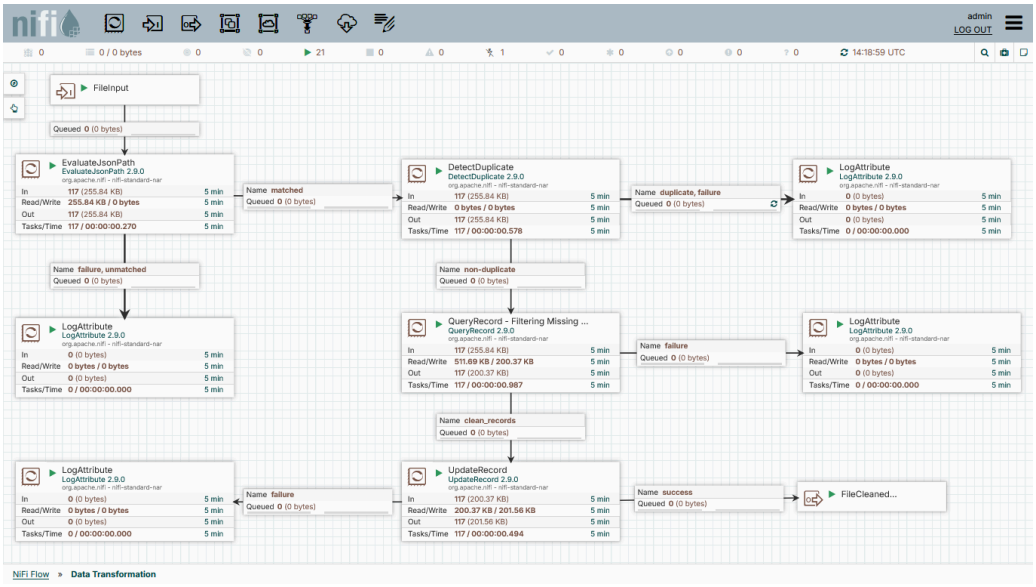

Figure 4: Data Transformation pipeline utilizing Record Readers/Writers.

- **Deduplication:** An `EvaluateJsonPath` processor extracts the `transaction_id`. A `DetectDuplicate` processor then checks this against NiFi's in-memory `MapCacheClientService` to filter out the 5% duplicate rate injected by the simulator. Duplicates are routed to a specific logging queue.
- **Filtering:** A `QueryRecord` processor uses standard SQL (`SELECT * FROM FLOWFILE WHERE price IS NOT NULL`) to dynamically drop invalid records.
- **Standardization:** An `UpdateRecord` processor converts the mix of Unix epoch integers and ISO strings into a clean, uniform `yyyy-MM-dd HH:mm:ss` timestamp format.

2.4 Phase 3: Database Ingestion (Incremental Extraction)

This group fulfills the requirement for an additional ingestion method (Database) and ensures strict incremental loading.

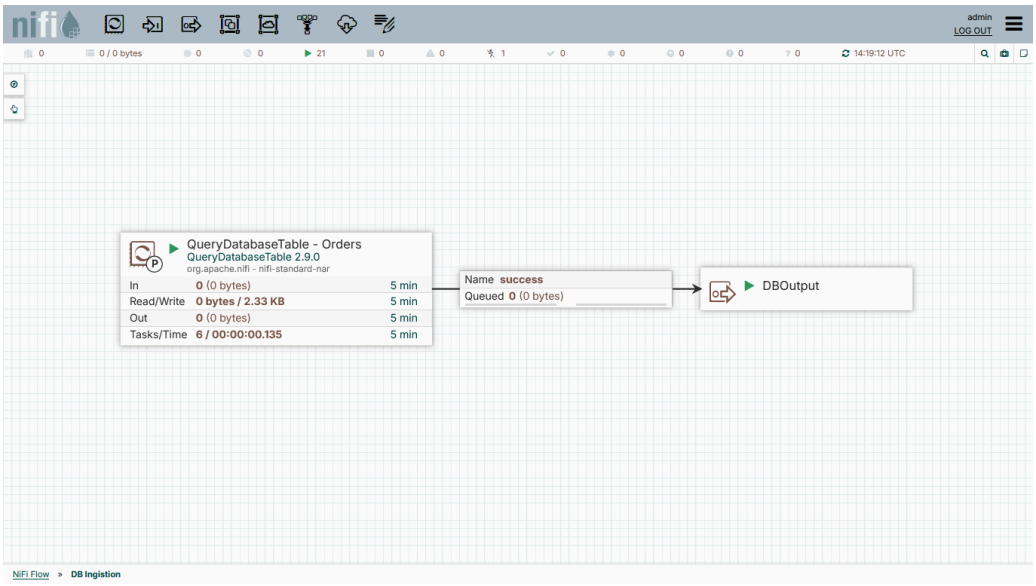

Figure 5: Database Ingestion via DBCPConnectionPool.

- **Design Decision:** The `QueryDatabaseTable` processor connects to the PostgreSQL operational database via a connection pool. It is configured to track the `updated_at` column as its Maximum-Value Column. This ensures that on every run, NiFi only extracts new or modified rows, avoiding expensive full-table loads.

2.5 Phase 4: Enrichment & HDFS Loading

This final group merges the two distinct data streams and writes them to the distributed file system.

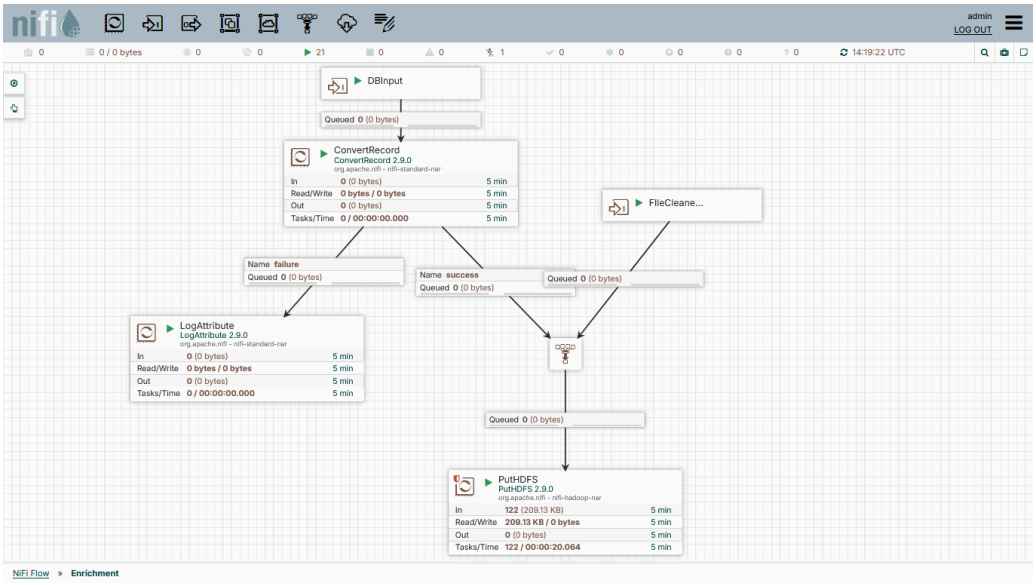

Figure 6: Enrichment and final HDFS append routing.

- **Format Conversion:** Because the database outputs records natively in Avro format, a `ConvertRecord` processor translates the database stream into JSON to perfectly match the file ingestion stream.
- **HDFS Storage:** The streams are merged into a `PutHDFS` processor configured with the `core-site.xml` and `hdfs-site.xml` Docker networking resources. The conflict resolution strategy is set to `Append` to safely batch the real-time streams into consolidated files in the `/user/itversity/ecommerce/processed_transactions/` directory.

3 Best Practices & Evaluation Criteria Met

- **Proper Relationship Handling:** No failure relationships are auto-terminated. All `failure`, `unmatched`, and `permission.denied` queues are routed to `LogAttribute` processors acting as a **Dead Letter Queue (DLQ)**. This guarantees zero silent data loss and establishes an audit trail.
- **Back Pressure & Yield:** Connection queues (such as the one between `ListFile` and `FetchFile`) have Back Pressure thresholds established to prevent JVM `OutOfMemory` errors if downstream systems slow down. Processors connecting to external systems utilize Yield Durations to gracefully handle network timeouts.
- **Transformation Complexity:** Successfully implemented in-memory caching for deduplication and dynamic RecordPath functions for data type casting without external service dependencies.

4 Sample Data Before and After Transformation

To verify the integrity of the data pipeline, samples were extracted from the source generation directory and the final HDFS destination. The transformations successfully standardized date formats and filtered out malformed and duplicate records.

4.1 1. Raw Simulated Data (Before)

This data represents the raw output from the Python simulator (Newline Delimited JSON). Note the presence of microsecond precision timestamps, a record with a `null` price, and an exact duplicate transaction at the end.


```
{"transaction_id": "8e581c40-4494-4ed0-b306-df97fd794322", "user_id": 55029, "product_name": "Reactive
fresh-thinking knowledge user", "price": 762.19, "payment_method": "Crypto", "timestamp":
"2026-05-11T13:57:16.657792"}
{"transaction_id": "252e537e-487c-4c2d-b42d-d4b612ff7b97", "user_id": 2753, "product_name": "Innovative
homogeneous attitude", "price": null, "payment_method": "Crypto", "timestamp":
"2026-05-11T13:57:16.658234"}
{"transaction_id": "699efff5-5c95-4d9e-8c0f-d74c38260015", "user_id": 93249, "product_name": "Distributed
client-server circuit", "price": 980.67, "payment_method": "Crypto", "timestamp":
"2026-05-11T14:12:44.168773"}
{"transaction_id": "699efff5-5c95-4d9e-8c0f-d74c38260015", "user_id": 93249, "product_name": "Distributed
client-server circuit", "price": 980.67, "payment_method": "Crypto", "timestamp":
"2026-05-11T14:12:44.168773"}
```


4.2 2. Processed HDFS Data (After)

This data was extracted directly from HDFS after passing through the NiFi pipeline. The pipeline successfully filtered out the record with the null price, removed the duplicate entry, and cast all timestamps into a standard yyyy-MM-dd HH:mm:ss SQL-compliant format.


```
{"transaction_id": "8e581c40-4494-4ed0-b306-df97fd794322", "user_id": 55029, "product_name": "Reactive
fresh-thinking knowledge user", "price": 762.19, "payment_method": "Crypto", "timestamp": "2026-05-11
13:57:16"}

{"transaction_id": "699efff5-5c95-4d9e-8c0f-d74c38260015", "user_id": 93249, "product_name": "Distributed
client-server circuit", "price": 980.67, "payment_method": "Crypto", "timestamp": "2026-05-11 14:12:44"}
```